

Audit. Salvage. Torch. Rebuild.

Six months of building left real assets behind — and real technical debt. This is the agent that decides which is which before anything gets carried into the next build.

*Not going to salvage two conflicting repos by force — no headlocks. **Rip it out at the root before it chokes the sunlight.** Hundreds of hours of real work exist in the old repos; none of the rot that crippled the last build comes along with it.*

WHAT THIS AGENT DOES

Repo Audit & Salvage

Uploaded with the data, skills, and tools to: analyze the user's actual end goals and assets, audit existing repo code/assets for usability and salvage-worthiness, and decide — piece by piece — what's worth keeping and what isn't. No stowaways in the cargo. Every carried-forward pattern gets checked, not assumed clean.

NOT THE GOAL

"Build project" does not mean implementing the repos. It means building the **structure** — the environment, scaffolding, and attached tools — so the later orchestrator session isn't starting from zero when it goes to actually implement. That session decides whether to use what's provided, extend it, or swap it out.

1

Ingest. User's stated end goals, plus every existing repo/asset in scope. **Likely candidate: Horizons UI** — this is where the real conflicting-repo tension lives. NeuroOmni (167 commits, last touched Jun 16, origin remote unresolved) is **not** a salvage candidate in the hopeful sense — confirmed no novel aspect remains, it's outdated, stale infrastructure. Confirm the full scope with the user before auditing, don't assume this list is complete.

2

Audit. Assess each piece against usability and salvage-worthiness. Flag anti-patterns, dead ends, and anything structurally responsible for problems in the last build — name them specifically, don't gesture at "technical debt" vaguely.

3

Decide. Per piece: salvage (clean, carry forward) or torch (discard). Confirm every discard with the user before it happens — nothing gets demolished silently.

4

Rebuild the structure. Ground-up repo scaffolding, asset- and data-rich, tools attached — not implemented, just ready. Document what's salvaged vs. rebuilt and why, so the next session inherits the reasoning, not just the result.

Standing rule this agent inherits: flag old/contradictory content with its age, confirm before removing — never auto-delete. Salvage/torch decisions are exactly what that rule was written for.

LAUNCH PROMPT — PASTE INTO A NEW SESSION TO START THIS

You are the Repo Audit & Salvage agent for Operation Launchpad. Read NovA-Corpus/handoff-nova-corpus-2026-08-14.md for context first.

Job: ingest my stated end goals and every existing repo/asset I name (starting candidate: NeuroOmni — 167 commits, last commit Jun 16, origin remote unresolved, needs my decision on what it should point to or whether it's in scope at all). Audit each for usability and salvage-worthiness. Name anti-patterns and structural problems specifically — don't gesture at "technical debt" vaguely. For each piece, recommend salvage or torch, and wait for my confirmation before discarding anything — nothing gets deleted without me saying so first.

Once decided: build a ground-up repo structure — asset-rich, tool-attached, NOT implemented — ready for a later orchestrator session to implement into. That session gets to decide whether to use what you built, extend it, or swap it for something else. Do not force-merge conflicting repos.

SIDE NOTE — LICENSING, IF AESOP GOES OPEN-SOURCE

What "salvage" can legally mean

Checked against real license files, not memory. Most of the stack is clean: LocalAI, CrewAI, Orca, LangFlow, the Pocock fork, reverse-skill, Honey, and Prime Agent are all **MIT**; mem0, reasoning-bank, and graphify are **Apache-2.0**. Both permit copying code straight into AESOP's own repo, even under a different license, provided the original copyright notice stays attached to what was copied.

TWO EXCEPTIONS, CHECKED DIRECTLY

OR1 has no license file at all — default copyright applies, meaning no formal permission to

CC-BY has no license file at all — default copyright applies, meaning no formal permission to copy its source into a published repo. Fix costs nothing: ask the maintainer to add one.

Ringer is PolyForm Shield 1.0.0 with an explicit noncompete clause covering "products that provide functionality through different kinds of interfaces... even if free of charge" — since AESOP is itself a swarm orchestrator, baking Ringer's code in is a real violation, not a gray area. Renaming or wrapping it doesn't avoid this; the clause is about function, not code organization.

Reverse-engineering a schema or protocol is a different, safer thing than copying source —

copyright protects the specific code, not the idea. Rebuilding a data schema or API shape from *observed behavior* (what goes in, what comes out) rather than from reading and paraphrasing the original source is well-established practice (this is literally how Samba reimplemented SMB, and the reasoning courts endorsed in *Google v. Oracle* for API reimplementation). The caveat that matters: it has to come from watching what something does, not from translating its source line-by-line with different names — the latter can still infringe even without literal copying. Database schemas specifically are on the safest ground of all; they're treated closer to facts than creative expression.